Sequent: SOP-Driven, Physics-Verified Loco-Manipulation on a Simulated Humanoid

Jatin Sikka
The University of Texas at Austin
https://jatinsikka.github.io/sequent-g1/ · Code: https://github.com/jatinsikka/sequent-g1

Abstract

Factories run on written Standard Operating Procedures (SOPs), but operators report problems in their own words. We built Sequent, a system in which a simulated Unitree G1 humanoid reads such a report, retrieves the SOP that applies out of 1,012, maps its steps to robot skills, and executes them in one continuous MuJoCo simulation. A verifier checks every step against the simulator state instead of trusting the policy, so a step that fails is reported as failed. The retriever is a small transformer fine-tuned with a contrastive loss. On vague operator tickets it ranks the correct SOP first 57.7% of the time (5 seeds), against 35.7% for BM25 and 40.9% for the same model without fine-tuning, and it keeps most of this gain on SOPs and machine families held out from training. For the body we use controllers where the problem is geometric (walking, grasping) and reinforcement learning where contact makes modeling hard (pressing a button, pulling a lever). Neither RL contact skill is solved. The button-press policy learned to farm reward by pressing repeatedly; we stopped it by ending the episode after the first held press rather than fixing the reward. The lever succeeds about half the time. We also report six reward exploits found by inspecting rendered rollouts.

1 Introduction

Industrial plants document their responses to faults as written Standard Operating Procedures (SOPs). Operators report faults in their own words, for example "the dispenser coasted to a stop and the thermal light is red," without naming the procedure. A robot that maps such a report to an approved SOP and executes it would be useful without open-ended planning.

Recent language-driven robot systems use large language models to turn instructions into plans [1, 2, 3]. A freely generated plan, however, is not tied to an approved procedure, and success is often judged by the model's own report or by the policy's reward. In a plant, the plan should come from an approved SOP, and a step should count as done only if the physical state changed.

Our approach makes three choices. First, we retrieve rather than generate: a retriever trained on operator tickets selects an existing SOP, so the robot runs only approved procedures.

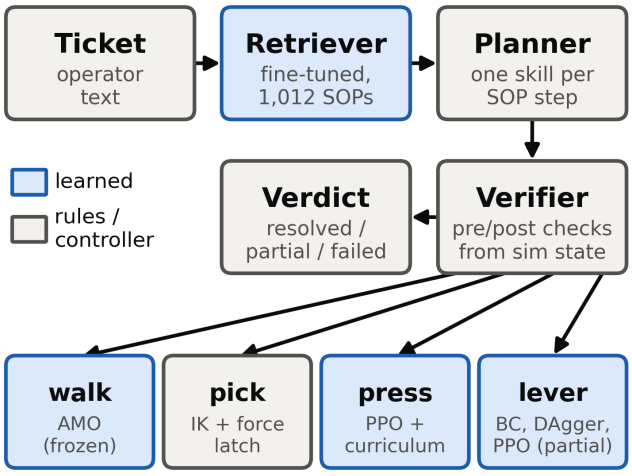


Figure 1: **System overview.** The fine-tuned retriever selects one of 1,012 SOPs, the planner maps each step to a skill by rule, and the verifier checks each skill's pre- and postconditions from simulator state. Blue: learned components. Grey: rules and controllers.

Second, we verify from simulator state: each skill runs inside a precondition and postcondition check that does not use the policy's output. Third, we use controllers for the geometric skills (walking, grasping) and RL for the contact-dominated ones (pressing, pulling).

Related work. Our work relates to four lines of research. *Language-conditioned task planning* uses LLMs to produce skill sequences, scored by affordances [1], written as code [2], or revised with feedback [3]. These plan freely; we restrict the robot to retrieved, pre-approved procedures, which trades flexibility for auditability. *Dense retrieval* trains bi-encoders on question–passage pairs [4, 5] and underlies retrieval-augmented generation [6]; lexical methods such as BM25 [7] remain strong baselines. We apply a standard bi-encoder recipe to a new setting, maintenance tickets against SOPs with many look-alike procedures, and measure where it fails. *Knowing when to ask* has been studied for LLM planners using calibrated uncertainty [8]; our retriever does not yet abstain, and we measure what a simple confidence cutoff would give. *Humanoid loco-manipulation and reward design*: we build on the AMO whole-body controller [9] for walking, and contrast with systems that learn manipulation from

demonstrations [10]. The reward hacks we document are instances of specification gaming [11, 12].

Contributions. In light of prior work, our contributions are:

1. A system that goes from an operator's ticket to a verified, multi-skill humanoid execution in one continuous simulation: walk, pick, carry, place, press, and a partial lever pull, with every step checked against simulator state.
2. A retrieval benchmark of 1,012 SOPs across 46 equipment families with 2,338 operator-style incidents, and a reproducible evaluation: the fine-tuned retriever reaches R@1 0.577 ± 0.006 versus 0.357 for BM25 and 0.409 untuned, with held-out-SOP, held-out-family, and ticket-clarity tests. Code, data, and per-run results are released.
3. Evidence for the controller/RL split: RL grasping on the gripper-equipped humanoid reached 0% verified success, while an IK controller with a force-gated latch held 11 of 12 lifts; PPO with a curriculum learned to reach and press the button, but also to farm reward by re-pressing, which we did not fix.
4. Six reward exploits observed during training, identified from rendered rollouts, and the fix applied to each (pumping was stopped by early termination, not fixed).

Organization. Section 2 states the problem. Section 3 describes the approach and the baselines. Section 4 reports retrieval, skill, and full-system results. Section 5 concludes.

2 Problem Statement

SOP corpus. A plant has a corpus $D = \{d_1, \ldots, d_N\}$ of $N = 1{,}012$ SOPs. Each SOP d has a title, a triggering condition, and an ordered list of steps $(\sigma_1, \ldots, \sigma_K)$. Each step names a skill from a fixed set $A = \{$`walk_to`, `pick`, `place`, `press_button`, `read_sensor`, `wait`, `notify`$\}$ and an entity, for example `press_button estop_button`.

Tickets. An operator writes a ticket q, free text such as "line 2 dispenser coasted to a stop, thermal indicator in the red." Each ticket has one correct SOP $d^*(q)$.

Retriever. An encoder E_θ maps text to a unit vector in $\mathbb{R}^{384}$, and the score of SOP d for ticket q is $s_\theta(q, d) = E_\theta(q)^\top E_\theta(d)$. The robot acts on $\hat{d}(q) = \arg\max_{d \in D} s_\theta(q, d)$.

Execution. A plan $\pi(\hat{d})$ is the sequence of skills in $\hat{d}$. The robot, with simulator state x_t, executes skill k with a controller or a learned policy. Each skill has a precondition $\mathrm{pre}_k(x_t) \in \{0, 1\}$ checked before it starts and a postcondition $\mathrm{post}_k(x_t) \in \{0, 1\}$ checked after it ends, both computed from simulator state (for example, "button displaced past threshold and held," "box above the table for a sustained window," "base upright"). The verdict is *resolved* if every postcondition holds and *partially resolved* or *failed* otherwise.

> **Problem 1 (retrieval).** Given training pairs $\{(q_j, d^*(q_j))\}$, find θ that maximizes $\Pr[\hat{d}(q) = d^*(q)]$ on held-out tickets, measured as Recall@1. We also report Recall@5 and mean reciprocal rank (MRR).
>
> **Problem 2 (verified execution).** Given $\hat{d}$, execute $\pi(\hat{d})$ on the humanoid so that each post_k holds, and report a verdict computed only from the post_k values, never from a policy's own success signal.

Difficulty. Tickets describe symptoms rather than procedures, and each machine family has about 22 SOPs that differ in small ways (pressure below setpoint, sudden collapse, above ceiling). The skills run in sequence on one body in one simulation, so errors propagate: arriving a few centimeters off makes the next reach harder.

3 Approach

3.1 Retriever

We fine-tune `all-MiniLM-L6-v2` [5] (22.7M parameters, 6 layers) on the 1,859 training pairs. Each SOP is represented as its title, condition, and steps in one string. Training uses the multiple-negatives ranking loss: for a batch of B pairs, each ticket should score its own SOP above the other $B - 1$ SOPs in the batch,

$$\mathcal{L}(\theta) = -\frac{1}{B} \sum_{j=1}^{B} \log \frac{\exp(\lambda\, s_\theta(q_j, d_j))}{\sum_{l=1}^{B} \exp(\lambda\, s_\theta(q_j, d_l))},$$

with $B = 32$ and $\lambda = 20$, for 3 epochs (about 2 minutes on a laptop). Hyperparameters are in the appendix.

Baselines. All are evaluated on the same index and test set.

- **TF-IDF**: unigram and bigram TF-IDF with cosine similarity.
- **BM25** [7]: Okapi BM25 over lowercase word tokens.
- **MiniLM-0**: the same encoder without fine-tuning. This isolates what training adds.
- **bge-small-0**: `bge-small-en-v1.5`, a stronger off-the-shelf encoder of similar size, without fine-tuning.
- **MiniLM-FT + CE**: our model with its top 10 reranked by the off-the-shelf cross-encoder `ms-marco-MiniLM-L-6-v2`.

3.2 Planner

The SOPs are written at skill granularity, so the planner is a deterministic mapping: one skill per written step, in order, with a `walk_to` inserted when the required location changes. Step arguments are mapped onto the simulator's objects (four colored buttons and a fixed set of tools). There is no learning in this step.

3.3 Skills

- **Walk**: the frozen AMO whole-body policy [9] with a closed-loop yaw servo. AMO cannot turn in place and

Table 1: **Retrieval on the held-out test set** (269 tickets, 1,012 SOPs). Fine-tuning adds 0.17 R@1 over the same encoder untuned. The cross-encoder reranker lowers R@1 and is not used.

Method	R@1	R@5	MRR
TF-IDF	0.301	0.535	0.408
BM25	0.357	0.569	0.461
MiniLM-0	0.409	0.677	0.529
bge-small-0	0.465	0.717	0.578
MiniLM-FT (ours)	**0.577 ± 0.006**	**0.856 ± 0.006**	**0.695 ± 0.005**
MiniLM-FT + CE	0.452 ± 0.006	0.843 ± 0.003	0.611 ± 0.004

drifts about 18° in yaw while walking, so the servo re-steers toward the target from the measured base pose.

- **Pick and place**: damped-least-squares IK to the box, then a latch that engages only when both gripper pads measure contact force, so a grasp cannot be declared across a gap.
- **Press**: PPO with a curriculum that moves the arm's start pose from touching the button to the true rest pose as the success rate rises.
- **Lever**: behavior cloning from scripted IK demonstrations, one DAgger pass, then PPO.

3.4 Verifier

The verifier wraps every skill with its precondition and post-condition (Section 2). It is independent of how a skill is implemented, so the same checks apply to controllers and learned policies. Before the final system existed, it already separated two grasp checkpoints with similar training curves: one with a claimed 30% success was 0% under verification (it grasped and never lifted), and one with a claimed 60% was 55%.

4 Experiments

Metrics. For retrieval: Recall@1 (the correct SOP is ranked first), Recall@5, and MRR over the full 1,012-SOP index. For skills and the full system: success as decided by the verifier.

4.1 Retrieval

Setup. The 269 held-out test tickets are ranked against all 1,012 SOPs. Fine-tuned rows are the mean and standard deviation over 5 seeds.

Generalization. Every test SOP also has training tickets, so the test above is closed-set. We add two splits: 20% of SOPs held out entirely (459 test tickets) and five equipment families held out (257 tickets). The fine-tuned model reaches R@1 0.656 and 0.607, against 0.468 and 0.420 for MiniLM-0 and 0.447 and 0.311 for BM25. Each split has its own test set, so only within-split comparisons are meaningful. The gain is therefore not limited to SOPs seen in training.

Table 2: **Skill results.** Pick uses a controller; press and lever use RL. The press and lever are partial, and RL grasping failed.

Skill	Method	Result	How measured
Pick	IK + force-gated latch	11 of 12 held lifts	fixed-base validation
Pick (in walk→pick run)	same	11.9 cm held lift	one continuous run
Press	PPO + curriculum	reaches from rest and presses; re-presses for reward unless stopped after the first hold	deterministic rollouts
Lever	BC → DAgger → PPO	1 of 2 completed throws	deterministic episodes
RL grasp, gripper humanoid	PPO, several reward variants	0% verified	deterministic evaluation

Ticket sets. We evaluate the same index on three ticket sets (Fig. 2): the held-out test tickets (R@1 0.577), the same tickets with the machine named (0.632), and 92 tickets written by the author from each sampled SOP's title and condition, two per family, without looking at model outputs (0.920; the correct SOP is in the top 5 for all 92). BM25 scores 0.357, 0.398, and 0.837. All methods score above 0.75 on the title-derived tickets; the fine-tuned model's margin is largest on the test tickets.

Error analysis. About 70% of errors on the test tickets select an SOP for a different machine. Naming the machine raises the share of top-1 picks on the correct machine from 0.71 to 0.85, and R@1 to 0.63, so most remaining errors are same-machine SOPs. In one seed, 4 of the 7 errors on the title-derived tickets selected another machine's SOP for the same symptom although the ticket named the machine. Only 1–3% of incorrect picks share the correct SOP's skill sequence, so most errors would execute a different procedure.

Selective retrieval. The retriever always acts on its top SOP. We measured the effect of a confidence cutoff on the gap between the top two scores, with the cutoff chosen on the validation split. Acting only above the cutoff covers 41% ± 3% of test tickets at 86% ± 3% precision, against 58% when acting on all of them; the remainder would be referred to a person. This is not part of the system.

4.2 Skills

RL grasping. RL grasping on the gripper-equipped humanoid reached 0% verified success, although the logged lift rate was 30–60%. Rendered rollouts showed the box being squeezed out of the pinch and moving upward, which trig-

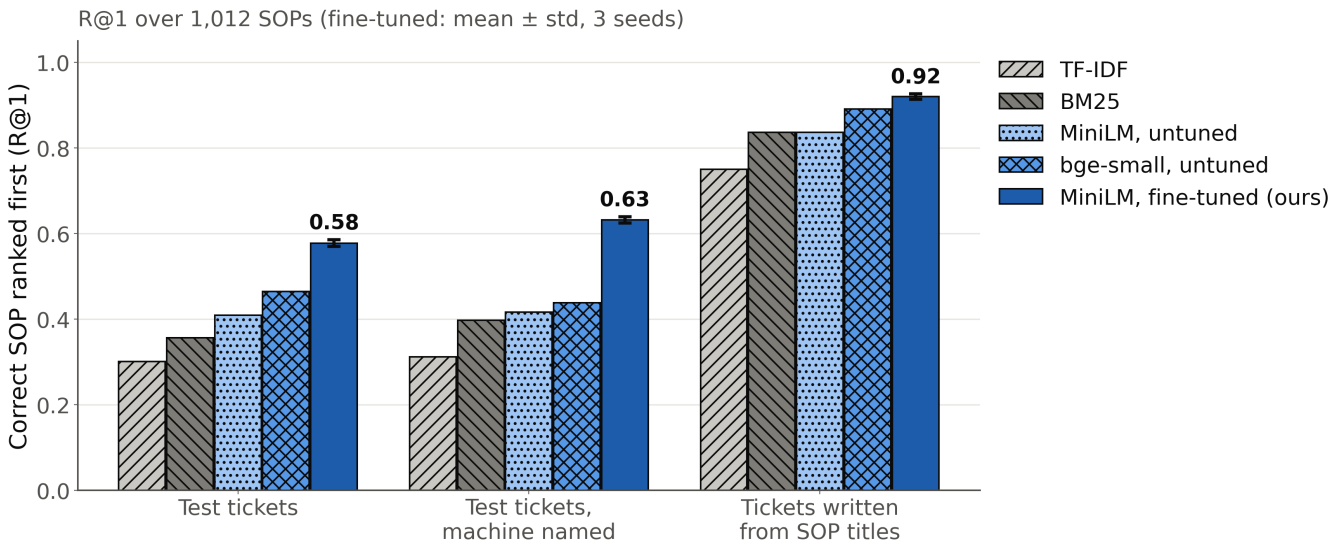


Figure 2: **R@1 by ticket set.** Held-out test tickets, the same tickets with the machine named, and 92 tickets written by the author from SOP titles. Fine-tuned bars: mean ± std over 3 seeds.

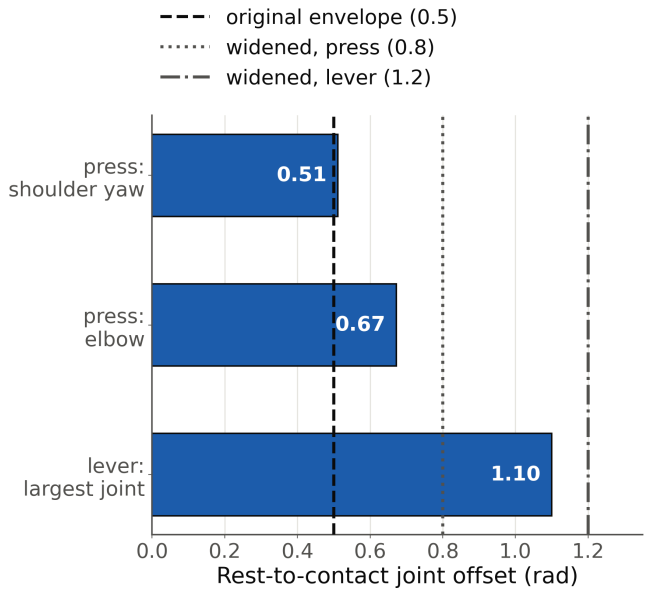


Figure 3: **Required joint offsets and action range.** Rest-to-contact joint offsets for the press and lever versus the policy's action range. The original ±0.5 rad range was smaller than every required offset, so far-start actions saturated at the clamp. The range was widened to 0.8 rad (press) and 1.2 rad (lever).

gered the height check without a grasp. A stochastic policy's grip force oscillates, and a stable two-sided pinch is a narrow regime in the contact solver. The IK controller with a force-gated latch held 11 of 12 lifts. RL grasping did reach 55% verified success on an earlier fixed-base rig with a different hand, so this result is specific to this embodiment and contact setup.

Action range. Far-start press training failed for a day because the policy's action range was 0.5 rad per joint around the neutral pose, while reaching the button from rest needs 0.511 rad at the shoulder and 0.672 rad at the elbow, and the lever needs up to 1.1 rad (Fig. 3). Saturated actions are not visible in the training curves.

Press. From the rest pose, random exploration never touched the button, so a contact-based reward gave no signal; a curriculum that starts the arm on the button and moves it back toward rest was needed to learn the reach at all. Two bugs cost time along the way: the action range was smaller than the reach required (Fig. 3), and a collision-mask error let the gripper pass through the buttons. Each reward change then produced a new exploit (Table 4). The last one, pumping, came from a per-step reward for press depth: pressing, releasing, and pressing again paid more than holding. We did not fix the reward. To run the full chain, the episode is stopped once the first press has been held for 0.6 s, so in the demos the policy never gets the chance to press again; left running, it does. We put the remaining time into the end-to-end system, the verifier, and the retriever. The fix we would make is to pay for one sustained press as a single event, penalize any release after it, and evaluate without early stopping.

Lever. The lever needed a latching mechanism so partial progress persisted, quieter exploration (initial log-std −1.5), and behavior cloning to reach any reward at all. The best policy completed the throw in 1 of 2 deterministic episodes, with jerky motion. The cause is identified: a per-episode reach offset that changes what an action does is not in the policy's observation, so identical observations call for different actions. The fix (add it to the observation) is designed but not trained.

4.3 Full system

Single-run chain. In one continuous simulation the robot walks to the table, picks the box, carries it to the panel with a 180° turn, places it, presses the button (34.5 mm; the run

Table 3: **Full run, best of three takes.** Verdict: "partially resolved." In the weakest take, place, press, and lever failed and the verifier reported each.

Step	Measured outcome	Verdict
pick	box gripped, lifted, held	pass
carry, place	carried to the panel, placed upright	pass
press	33.8 mm, held	pass
lever	0.443 rad; success requires < 0.25 rad (best of 4 stance variants; others 0.595 and 0.627)	fail
notify	verdict card sent	pass

stops the press policy after its first 0.6 s hold), and returns to rest. Every postcondition holds.

Ticket to robot. For the final run we wrote one SOP (SOP-1013) that uses all five embodied skills and added it to the corpus. The retriever ranks it first (score 0.897). Because the ticket and SOP were written together, and TF-IDF also ranks it first, this run shows the pipeline end to end rather than retrieval accuracy. Of three takes, the best is:

4.4 Reward hacks

Table 4: **Reward exploits observed in training,** each found by inspecting rendered rollouts. Pumping was stopped by early termination, not fixed. The other fixes fall into three classes: terminating what can be farmed, moving constraints from the reward into the controller, and checking sustained states rather than instantaneous ones.

Reward term	What the policy did	Fix
Proximity income near the button	Hovered 5 cm short and collected it indefinitely	Narrow the band from 0.20 to 0.05 m, double the distance penalty
Progress along the lever arc	Sat in the lever's 0.17 rad of free travel	Measure progress from past the free travel, make it convex
Per-step press depth	Pressed, released, pressed again	Not fixed: the episode is stopped after a 0.6 s hold; left running, the policy re-presses
Hold counter after the press	Swung the arm while the counter ran	Stillness cost after the press; hold counter tolerates brief dips
Jerk penalty	Flailed fast enough that bumping the button paid more than the penalty (jerk 0.83)	Low-pass filter on the arm command, $\alpha = 0.12$ (jerk 0.001)
Lift height	Launched the box out of a crushing pinch	Replace RL with a controller; verify a sustained held state

Limitations. Everything runs in MuJoCo; no hardware transfer was attempted. The SOPs and tickets come from one generator, and there is no evaluation on real maintenance tickets. Because SOPs are written at skill granularity, planning is a lookup, and SOPs written in ordinary prose would need a learned planner. The retriever cannot decline. Skill results are small-sample. The press policy still re-presses for reward when it is not stopped, and the lever succeeds about half the time. The reward hacks were judged by one reviewer.

5 Conclusion

Sequent maps an operator's ticket to an approved SOP and executes it on a simulated humanoid, with a verifier that reports "partially resolved" when a step fails. The fine-tuned retriever reaches R@1 0.577 on the held-out test tickets and 0.92 on title-derived tickets, and its gain over the untuned model holds on held-out SOPs and equipment families. On the body, controllers outperformed RL for grasping on this embodiment. The RL press learned to reach and press but also to re-press for reward, which we stopped rather than fixed; the lever is partial. The reward exploits were identified from rendered rollouts, not from training metrics.

Future work. The most important addition is for the retriever to ask instead of guess, using the confidence measured above, and to filter by the machine ID that a real ticket carries. Beyond that: hard-negative training against same-machine look-alike SOPs; a comparison against an LLM choosing among the retriever's candidates; a learned planner for SOPs written in prose; retraining the press so it stops on its own (reward one sustained press as an event, train without early stopping); retraining the lever with the missing observation; moving the body to Isaac Lab for faster contact training; and a test set of real tickets. Code, data, and results: https://github.com/jatinsikka/sequent-g1.

Acknowledgments

The AMO whole-body controller is by Li, Cheng et al. [9] and is used frozen. Training ran on Azure virtual machines.

References

[1] M. Ahn et al. Do As I Can, Not As I Say: Grounding Language in Robotic Affordances. In *Conference on Robot Learning (CoRL)*, 2022.

[2] J. Liang et al. Code as Policies: Language Model Programs for Embodied Control. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2023.

[3] W. Huang et al. Inner Monologue: Embodied Reasoning through Planning with Language Models. In *Conference on Robot Learning (CoRL)*, 2022.

[4] V. Karpukhin et al. Dense Passage Retrieval for Open-Domain Question Answering. In *Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[5] N. Reimers and I. Gurevych. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In *Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[6] P. Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[7] S. Robertson and H. Zaragoza. The Probabilistic Relevance Framework: BM25 and Beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[8] A. Z. Ren et al. Robots That Ask For Help: Uncertainty Alignment for Large Language Model Planners. In *Conference on Robot Learning (CoRL)*, 2023.

[9] J. Li, X. Cheng, T. Huang, S. Yang, R.-Z. Qiu, and X. Wang. AMO: Adaptive Motion Optimization for Hyper-Dexterous Humanoid Whole-Body Control. In *Robotics: Science and Systems (RSS)*, 2025.

[10] J. Li et al. OKAMI: Teaching Humanoid Robots Manipulation Skills through Single Video Imitation. In *Conference on Robot Learning (CoRL)*, 2024.

[11] D. Amodei et al. Concrete Problems in AI Safety. arXiv:1606.06565, 2016.

[12] J. Skalse et al. Defining and Characterizing Reward Hacking. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[13] J. Schulman et al. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.

[14] S. Ross, G. Gordon, and D. Bagnell. A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning. In *AISTATS*, 2011.

[15] E. Todorov, T. Erez, and Y. Tassa. MuJoCo: A Physics Engine for Model-Based Control. In *IEEE/RSJ IROS*, 2012.

Appendix

Retriever. `sentence-transformers/all-MiniLM-L6-v2`, mean pooling, 384-dimensional output, maximum sequence length 256. Multiple-negatives ranking loss with cosine similarity and scale $\lambda = 20$. Batch 32, 3 epochs, AdamW with learning rate 2×10^{-5}, linear warmup over the first 10% of steps. Seeds 0–4 for Table 1 and 0–2 for the other experiments. One epoch gives R@1 0.539 ± 0.013 (3 seeds).

Data. 1,012 SOPs, 46 families × 22 SOPs. 2,338 incidents split 1,859 / 210 / 269 (train / validation / test); the split puts each SOP's first incident in training, so every test SOP is seen in training. Held-out-SOP split: 20% of SOPs sampled with seed 0, all their incidents used for test. Held-out-family split: five families sampled with seed 0 (air compressor, filling machine, laser cutter, mechanical press, press brake). No test ticket has a near-duplicate in training (highest TF-IDF cosine 0.31). Informed tickets: two SOPs per family sampled with seed 2026, one ticket each.

Confidence cutoff. Signals: the top-1 score, and the gap between the top-1 and top-2 scores. For each target precision (0.80, 0.90, 0.95) the cutoff is the lowest one that meets the target on the validation split; it is then applied to the test split.

PPO [13]. Stable-Baselines3, MLP policy with two hidden layers of 128 units, `n_steps` 1024 per environment, batch 1024, 10 epochs per update, learning rate 3×10^{-4}, $\gamma = 0.99$, 200-step episodes. Entropy coefficient 0.02 (press) and 0.05 (lever, followed by a low-entropy phase). Action range 0.8 rad (press) and 1.2 rad (lever) around the neutral pose; arm command low-pass filter $\alpha = 0.12$. Lever: initial log-std −1.5, 33-dimensional observation. Training used 24–56 parallel environments on 32- and 64-core Azure VMs. With default threading, parallel workers each spawned a full set of torch threads and throughput fell to 18 steps/s; setting `OMP_NUM_THREADS=1` per worker restored about 2,600 steps/s.

Simulator. MuJoCo 3.2.3 [15], 29-DoF Unitree G1 with a Robotiq 2F-85 gripper on the right wrist, AMO at 50 Hz.

Reproducing. `brain/benchmark/REPRO_REPORT.md` lists every retrieval number with the script and CSV that produced it. RL checkpoints are not released.